

United International University

School of Business and Economics

Real-Life Project: Predictive Analytics

Stock Price Prediction, Sales Forecasting, and Waiter Tips Regression

Name: Hussain Md Sayeam Student ID: 905253014 Group Name: Four AngryMen

Report: Project Summary & Result Interpretation

Project brief

Building a comparative predictive models across three problem types: a time-series neural network (LSTM) for stock prices, a classical time-series forecast for sales, and a refined regression model for waiter tips. Also showed up the package one of my models into a deployable prediction pipeline. There have been described the findings of the project as well.

LOAD of Dataset

A given reference file of CSV format on “predictive_analytics_dataset.xlsx” — three sheets: Stock_Prices (three tickers, ~750 trading days each, OHLC + volume), Sales_Data (four stores, 730 daily sales figures with trend and seasonality), and Waiter_Tips (300 restaurant bills with tip, party size, day, time, and demographics); showing an LSTM stock predictor, Holt-Winters sales forecasting, waiter tips regression refinement, and a saved, reloadable prediction pipeline. Use it as a reference, not as your submission.

Result Interpretation and Report

(1) Executive Summary of Model Performance

This project developed three distinct analytical solutions. First, an LSTM neural network was trained on TECHCORP stock prices. By utilizing a 20-day sliding window, the model learned sequential dependencies and achieved a Root Mean Squared Error (RMSE) of ~\$4.83 on the unseen test period. Second, a Holt-Winters Exponential Smoothing model was applied to the North store's sales data. By factoring in additive trends and a 7-day seasonal period, it achieved an RMSE of ~\$93.41. Finally, a restaurant tipping regression model was refined through three iterations.

Time-Based vs. Random Train/Test Splitting For the waiter tips dataset, a standard randomized train/test split was used because each restaurant bill is a cross-sectional, independent event. However, applying a random split to the stock or sales data would induce massive data leakage. Time-series forecasting relies on the premise of using the past to predict the future. If we shuffle time-series data, the model might learn from "future" data points to predict a "past" data point. Therefore, a strict chronological split (first 80% for training, last 20% for testing) was strictly adhered to for the LSTM and Holt-Winters models to ensure the evaluation reflects true forecasting conditions.

(2) Waiter Tips Model Refinement and Insights

The baseline Linear Regression model, relying solely on `total_bill`, achieved an R^2 of 0.5031 and an RMSE of 1.0240.

- **Refinement 1 (Ridge Regression):** By incorporating categorical features (sex, smoker, day, time) and party size through a preprocessing pipeline, the Ridge model improved the R^2 to 0.5315.
- **Refinement 2 (Random Forest):** The non-linear Random Forest achieved the best results with an R^2 of 0.5760 and an RMSE of 0.9459.

Insight: The jump in accuracy from the baseline to the Random Forest indicates that while `total_bill` is the strongest predictor, tip size is heavily influenced by non-linear interactions between categorical variables—for example, large party sizes might tip at higher percentages specifically on weekend dinners compared to weekday lunches.

Deployment Choice: I would deploy the **Random Forest Pipeline**. While slightly less interpretable than Ridge Regression, it yields the highest predictive accuracy. Because the feature space is small (only 6 features) and the data is tabular, the risk of severe overfitting is mitigated by setting proper `n_estimators`, making it robust for production.

(3) Limitations and Model Improvements

1. **Limitation (Univariate Constraints):** Both the LSTM and Holt-Winters models currently only look at historical values of the target variable. *Improvement:* For the stock model, incorporating exogenous variables like trading volume, moving averages, or macroeconomic indicators could vastly improve predictive power.

2. **Limitation (Data Size/Scope):** The models were trained on a single ticker and a single store over a short synthetic history. *Improvement:* Training a generalized, global forecasting model (e.g., XGBoost or a larger Deep Learning architecture) across all stores or tickers simultaneously would allow the model to learn cross-sectional behaviors and handle anomalies better.

(4) Deployment Architecture: Batch vs. On-Demand

Deploying the three models requires different architectural approaches:

- **On-Demand (API-based):** The **Waiter Tips** model should be deployed as a REST API (e.g., using FastAPI or Flask). As a waiter enters a bill into the Point of Sale system, the system makes a real-time request to the API, which instantly returns the expected tip.
- **Scheduled Batch Jobs:** The **Sales** and **Stock Price** models do not require real-time, second-by-second updates. They should be deployed as daily scheduled batch jobs (e.g., using Apache Airflow or cron). Every night at midnight, a pipeline would ingest the day's closing data, generate the forecast for the next 24 hours, and push the predictions to a dashboard for managers to review in the morning.